A Multi-Backend FFT-NILT Library for Spectral Slab PDE Solvers

GORGI PAVLOV, Lehigh University, USA

We present SCALPEL, an open-source Python library that ships a single batched FFT-accelerated numerical inverse Laplace transform (NILT) primitive as a reusable kernel for spectral slab PDE solvers across four physics domains (lossy Maxwell, preparative chromatography, thermoviscous acoustics, and elastodynamics) and a fractional Caputo subdiffusion stress test. The library targets a gap in scientific Python: no portable, multi-backend FFT-NILT primitive currently exists that runs unchanged across three Array-API-conformant frontends (NumPy, PyTorch, JAX) on both CPU and GPU under a single dispatcher, with built-in finite-precision parameter auto-tuning, and with CuPy and Julia covered through reference wrappers for portability validation.

The algorithmic core is a Dubner-Abate FFT inversion of modewise transfer functions $H(s, k_\perp, d) = \exp(-d\,\gamma_z(s, k_\perp))$ at $O(N_x N_y N_{\mathrm{NILT}} \log N_{\mathrm{NILT}})$ cost, batched over all transverse Fourier modes and observation times in a single inverse FFT. The library's parameter-selection module is backed by two closed-form finite-precision feasibility estimates, distinct in both mechanism and scaling: for telegrapher-type slabs ($\gamma_z^2 = \alpha s + \beta s^2 - k_\perp^2$) the rightmost branch point sits in the open right half-plane and imposes a positive lower bound on the Bromwich shift, yielding a time-limited feasibility boundary $k_\perp^{\max} \propto \sqrt{L/t_{\max}}$ to $L/t_{\max}$ depending on regime; for diffusion-type slabs ($\gamma_z^2 = A + Bs + k_\perp^2$) all branch points lie in the closed left half-plane and the operative estimate is direct transfer-function underflow at the contour origin, giving $k_\perp^{\max} \sim L_{\mathrm{eff}}/d$. The two estimates ship in SCALPEL as the `feasibility` module; users do not tune Bromwich parameters by hand.

We report (i) an eight-backend × device wall-clock benchmark on a single workstation with a consumer GPU spanning 8–153× speedup over 3D Yee FDTD [19] for the telegrapher class (high end attained by PyTorch CPU, GPU rows 27–121×) and 12–394× over an explicit-time FTCS+L1 baseline for fractional Caputo (high end attained by CuPy GPU, remaining GPU rows 19–300×); per-row data in `data/benchmark_multi_backend.csv`. (ii) a transform-domain head-to-head showing the batched FFT-NILT is ∼ 5× faster than an optimized de Hoog inversion and ∼ 13× faster than fixed Talbot at matched accuracy for the dense-grid, many-mode workload, confirming the advantage reflects FFT amortization across modes rather than mere comparison against explicit time marching; and (iii) an mpmath 50-digit Talbot ground-truth reference confirming float64 accuracy to 8×10^{-4} relative error at the validation depth. The complete source, raw benchmark data (CSV), reproducibility scripts, and a regression test suite spanning all four backends and four physics domains are deposited at doi:10.5281/zenodo.20682437 under the Apache-2.0 license. We request consideration for ACM *Artifact Available* badging.

CCS Concepts: • **Mathematics of computing** → **Numerical analysis**; **Numerical analysis**; • **Computing methodologies** → *Parallel computing methodologies*; • **Theory of computation** → *Design and analysis of algorithms.*

Additional Key Words and Phrases: scientific software, numerical inverse Laplace transform, spectral methods, GPU computing, multi-backend portability, reproducibility, finite-precision arithmetic, Dubner-Abate inversion, slab geometry, dispersion classes, fractional Caputo diffusion

1 Introduction

The scientific Python ecosystem has matured around array-language backends with broadly compatible APIs — NumPy, PyTorch, JAX, Julia (through PyJulia or native) — and consumer GPUs are now routine research hardware. Yet for one specific algorithmic primitive that arises in dozens of physics codes — a batched, FFT-accelerated numerical inverse Laplace transform (NILT) operating on modewise transfer functions of the form $H(s, k_\perp, d) = \exp(-d\,\gamma_z(s, k_\perp))$ — the existing software landscape consists of either single-backend, single-precision scripts buried inside larger solvers, or

Author's Contact Information: Gorgi Pavlov, Lehigh University, Bethlehem, PA, USA, gop214@lehigh.edu.

2026. Manuscript submitted to ACM

general-purpose serial NILT libraries (e.g., `mpmath` [10] Talbot inversion, `InverseLaplace.jl`) that are accuracy-first and not designed for the $O(N_x N_y N_{\mathrm{NILT}} \log N_{\mathrm{NILT}})$ throughput regime that GPU workflows demand. There is no portable, backend-agnostic, GPU-aware, finite-precision-auto-tuning FFT-NILT primitive that a downstream solver can drop in without rebuilding the parameter selection by hand.

SCALPEL (`spectral-scalpel`) closes that gap. The library exposes a single primitive that:

(1) Inverts modewise Laplace-Fourier transfer functions in batched form across all transverse modes and observation times via one inverse FFT, eliminating both the propagation-direction marching loop and the CFL-bound time loop that explicit solvers (e.g., Yee FDTD, FTCS) carry.
(2) Runs unchanged across NumPy, PyTorch, JAX, and Julia on either CPU or GPU via a thin dispatcher built on top of the Array API ([4]).
(3) Auto-selects Bromwich-contour parameters from class-dependent finite-precision feasibility estimates — two closed-form bounds, one per dispersion class, derived in Section 4 and shipped as the `feasibility` module.
(4) Plugs in physics-specific dispersion relations $\gamma_z(s, k_\perp)$ from a registry: lossy Maxwell, preparative chromatography, thermoviscous acoustics, P/S elastodynamics, and a fractional Caputo single-term pencil are bundled out of the box; adding a new dispersion class is a single Python file.

The primitive is benchmarked end-to-end against established explicit-time references (3D Yee FDTD [15, 19] for telegrapher kernels, FTCS+L1 for fractional Caputo) across eight backend × device combinations on a single workstation equipped with a consumer GPU (NVIDIA RTX 5060), and head-to-head in the transform domain against two scalar NILT alternatives (de Hoog's quotient-difference scheme and the fixed Talbot contour). Accuracy is anchored against a `mpmath` 50-digit Talbot inversion of the same modewise transfer function.

1.1 Why a library, not a script

The constituent pieces of the library — the Dubner-Abate trapezoidal NILT, the slab transfer function, the modewise Parseval error estimate, the de Hoog comparison — are individually well known. What is not common is having them composed into a single, batched, multi-backend, GPU-aware primitive with built-in finite-precision parameter selection that downstream solvers can use without re-deriving the Bromwich-shift tuning rules and without losing portability. We sustained four physics-domain plugins (Maxwell, chromatography, acoustics, elastodynamics) plus a fractional Caputo stress test through a single API for the duration of this study; the only per-domain code is the dispersion-class plugin (typically 80–150 lines per domain). That cross-domain reuse is the library's principal value proposition, and it is what distinguishes SCALPEL from the typical research-grade NILT script buried in a single-domain solver.

1.2 Contributions of this paper

This paper presents the library, its algorithmic core, and its validation:

(C1) **The SCALPEL library** (Section 2), an open-source Apache-2.0 Python package whose public surface comprises twenty-six top-level names across six modules (fifteen functions plus eleven dataclasses with constructor-callable semantics): five bundled physics propagators (`scalpel.api`: Maxwell, chromatography (Cartesian and Hankel paths), acoustic, elastodynamic), the memory-bounded chunked dispatcher (`scalpel.chunked`), an introspection / diagnostics layer that exposes the auto-tuner's decisions (`scalpel.diagnose`), a parameter-sweep harness (`scalpel.sweep`), a validation API for matched-precision comparison against external references

(`scalpel.validate`), and a configuration serializer for reproducible runs (`scalpel.config`). The dispersion-class plugin registry is the extension point for new physics.

(C2) **Algorithmic specification of the FFT-NILT primitive** (Section 3): the Dubner-Abate inversion as a single batched inverse FFT over all transverse modes and observation times, with explicit memory layout and dispatch.

(C3) **Class-dependent finite-precision parameter selection** (Section 4), the mathematical backing for the library's `feasibility` auto-tuner: two closed-form bounds on the largest transverse wavenumber the method can resolve at finite precision, derived from the branch-cut structure of γ_z, with distinct scalings for telegrapher-type and diffusion-type slabs. These are formally proved (Theorems 4.2 and 4.3) and ship to users as a single API call.

(C4) **Multi-backend implementation and portability evidence** (Section 5): the library runs on NumPy, PyTorch, JAX, and Julia, on both CPU and GPU; we describe the dispatch architecture and the JAX CPU-vs-GPU subprocess sidecar required to benchmark both devices in one process.

(C5) **End-to-end benchmarks and reproducibility** (Section 6): wall-clock comparisons against established explicit-time references, in-transform-domain comparison against de Hoog and fixed Talbot at matched accuracy, mpmath ground-truth validation, and the complete reproducibility chain.

(C6) **ACM Artifact Available compliance** (Section 7): Apache-2.0 license, Zenodo DOI, CITATION.cff, regression test suite with seeds, and a one-command reproduction script.

1.3 Position relative to existing software

Existing scientific-Python NILT packages include `mpmath`'s Talbot inversion (arbitrary precision, scalar, no GPU), `InverseLaplace.jl` (Julia, scalar, no batched FFT path), and research codes published with individual papers (typically NumPy- or MATLAB-only, single-domain, no parameter auto-tuning). None offer the combination of multi-backend portability, batched FFT operation, and finite-precision auto-tuning that SCALPEL provides. For the spectral-acoustic application specifically, *k*-Wave [17] provides a feature-rich CPU/GPU pseudospectral time-domain solver but operates on the time grid with CFL constraints rather than in the Laplace domain. For Maxwell, the FDTD ecosystem (`meep` [12], `Tidy3D`) is similarly time-grid-based; transform-domain approaches such as the angular spectrum method [8] typically operate frequency-by-frequency. SCALPEL is, to our knowledge, the first library to expose a batched FFT-NILT primitive as a portable scientific-Python module across these and adjacent physics domains.

1.4 Outline

Section 2 presents the SCALPEL library architecture and API. Section 3 specifies the FFT-NILT algorithm. Section 4 states and proves the two finite-precision feasibility theorems backing the parameter auto-tuner. Section 5 describes the multi-backend implementation. Section 6 reports the end-to-end and transform-domain benchmarks. Section 7 describes the reproducibility infrastructure and ACM Artifact Available compliance. Sections 8 and 9 catalog limitations and conclude.

2 The SCALPEL library

2.1 Architecture

The library is organized into eleven runtime modules (under `scalpel/`) plus the test suite (`tests/`) and the CI workflow (`.github/workflows/`); see Table 1. The runtime split follows the forward-map workflow: a physics domain (`scalpel.systems`) produces a dispersion relation (`scalpel.core.dispersion`) that the NILT engine (`scalpel.core.nilt`)

inverts, with parameters auto-selected by the feasibility module (`scalpel.core.feasibility`) and array operations dispatched through the backend layer (`scalpel.backends`). Above the engine, five public utility modules give users single-call access to chunked dispatch (`scalpel.chunked`), diagnostics (`scalpel.diagnose`), parameter sweeps (`scalpel.sweep`), reference-comparison validation (`scalpel.validate`), and reproducible run-configuration save/load (`scalpel.config`).

Table 1. The SCALPEL library: module layout and approximate line counts (Python only, excluding tests). The first block is the algorithmic core; the second is the user-facing API surface; the third is reproducibility infrastructure that is not part of the runtime API.

Module	Role
Algorithmic core	
`scalpel/core/`	NILT engine, dispersion registry, feasibility auto-tuner, Hankel transform
`scalpel/backends/`	Dispatched backends (NumPy default, PyTorch, JAX)
`scalpel/systems/`	Physics dispersions (Maxwell, chromatography, acoustics, elastodynamics)
User-facing API surface (26 top-level callables: 15 functions + 11 dataclasses)	
`scalpel/api.py`	Bundled physics propagators: `propagate_maxwell`, `propagate_acoustic`, `propagate_chromatography` (Cartesian path
`scalpel/chunked.py`	Memory-bounded tiled dispatch over the transverse grid
`scalpel/diagnose.py`	Introspection / auto-tuner-decision reports
`scalpel/sweep.py`	Cartesian-product parameter sweep harness
`scalpel/validate.py`	Matched-precision comparison against external references
`scalpel/config.py`	JSON serialization for reproducible run configurations
Reproducibility support (not runtime API)	
`scalpel/reference/`	Reference solvers (mpmath 50-digit Talbot, RK45, FDTD references)
`scalpel/benchmarks/`	Wall-clock and accuracy harnesses
`tests/`	pytest regression suite (incl. `test_api_expansion.py`, `test_chromatography_hankel.py`)
`.github/workflows/ci.yml`	Multi-OS / multi-Python / per-backend CI matrix

2.2 Public API

The library exposes twenty-six top-level callables across six modules (fifteen functions plus eleven dataclasses). The most common workflow is a single propagation call:

```
import scalpel as sc

field, t = sc.propagate_maxwell(
    source, depth=0.5,
    material=sc.MaxwellParams(sigma=1e-3, epsilon_r=4.0),
    grid=grid, nilt=nilt,
)
```

For chromatography specifically, the naturally cylindrical geometry suggests a quasi-discrete Hankel transform [9] in r rather than the Cartesian (K_X, K_Y) FFT shortcut. The `propagate_chromatography_hankel` variant samples the radial direction at scaled Bessel zeros and runs the convective substitution $v/(2D_z)$ automatically:

```
from scalpel.core.hankel import HankelTransform
ht = HankelTransform(R=0.005, N=64)            # column radius 5 mm
source_r = gaussian(ht.r, sigma=ht.R / 4)
field, t = sc.propagate_chromatography_hankel(
    source_r, depth=0.05, R=ht.R,
```

```
    column=sc.ChromatographyParams(v=1e-3, Dz=1e-7, Dr=1e-9),
    nilt=nilt,
)
```

A power user wanting reviewer-grade introspection on what the auto-tuner decided uses `scalpel.diagnose` instead:

```
field, t, report = sc.diagnose.run_and_report(
    physics='maxwell', source_xy=source,
    depth=0.5, t_max=20e-9, n_nilt=2048,
    precision='float64', grid=(128, 128, 0.1),
    reference='mpmath',
)
print(report.summary())          # Bromwich shift, contour params, theory kpmax,
                                 #   grid kpmax, margin dB, accuracy report
```

For grids larger than VRAM, `scalpel.chunked` composes the primitive over transverse tiles sized to a memory budget:

```
field, t = sc.chunked.batched_forward_chunked(
    sc.propagate_maxwell, source, depth, material, grid, nilt,
    vram_budget_bytes=int(7e9),
    progress=lambda done, total: print(f'tile {done}/{total}'),
)
```

For parameter studies, `scalpel.sweep` runs a Cartesian product over user-supplied varying parameters:

```
results = sc.sweep.parameter_sweep(
    forward_callable=sc.propagate_maxwell,
    varying={'depth': [0.1, 0.5, 1.0]},
    fixed={'source_xy': source, 'material': material,
           'grid': grid, 'nilt': nilt},
    reduce='rms',
)
results.to_csv('depth_sweep.csv')
```

For accuracy validation, `scalpel.validate` runs the primitive and a high-precision reference at matched parameters and returns a pass/fail report:

```
verdict = sc.validate.against_mpmath(
    physics='fractional', alpha=0.7,
    depth=0.1, t_max=20e-3,
    n_nilt=2048, precision='float64', tolerance=1e-3,
)
assert verdict.passed, verdict.summary()
```

Finally, `scalpel.config` round-trips the auto-tuned configuration to JSON so a later run reproduces the prior choices:

```
sc.save_run_config('run.json',
    dispersion='maxwell',
    dispersion_params={'sigma': 1e-3, 'epsilon_r': 4.0},
    auto_tuner_choices={'a': 1.0, 'T': 20e-9, 'N': 2048},
    grid={'Nx': 128, 'Ny': 128, 'dx': 0.1},
    precision='float64', backend='jax_gpu',
```

```
)
cfg = sc.load_run_config('run.json')
```

The six modules above decouple *what physics* (dispersion plugin), *what precision/depth/time domain* (feasibility auto-tuner), *how the inversion runs* (batched FFT-NILT, optionally chunked), *which backend and device* (dispatcher), *whether the run is auditable* (diagnose, validate, config), and *how it sweeps over parameters* (sweep). All twenty-four `test_api_expansion.py` tests for these modules pass under `pytest -q` (validated in CI; see Section 7).

2.3 Adding a new dispersion class

A user with a new physics application adds one file that defines the modewise transfer function. The minimum interface is:

```
@sc.dispersion.register('my_pde', dispersion_class='telegrapher')
def my_dispersion(s, k_perp, **params):
    """Return gamma_z(s, k_perp) for the PDE class."""
    return sc.sqrt(params['A'] + params['B']*s
                   + params['chi']*k_perp**2)
```

Once registered, `my_pde` flows through the auto-tuner and the NILT engine without further intervention. The feasibility module selects the appropriate finite-precision bound from the `dispersion_class` tag on the decorator: `'telegrapher'` routes to Proposition 4.2; `'diffusion'` routes to Proposition 4.3; `'fractional'` routes to the diffusion bound at the user-supplied `alpha`. Dispersions registered without a class tag fall back to a conservative default (low $k_\perp^{\max}$, modest N_{NILT}) that produces correct results at the cost of throughput; the API logs a warning. The four bundled physics classes (Section 2.4) all carry explicit `dispersion_class` tags.

2.4 Bundled physics dispersions

Four physics classes ship with the library (Table 2):

Table 2. Physics dispersions bundled in `scalpel.systems`, with the dispersion-class category each falls into.

Module	Physics
`maxwell.py`	Lossy electromagnetic slab
`chromatography.py`	Preparative-scale mass transport (Cartesian path uses $K_R = \sqrt{K_X^2 + K_Y^2}$; a Hankel-mode path [9] on the natural cylindrical grid is
`acoustics.py`	Stokes-corrected thermoviscous acoustics
`elastodynamics.py`	P/S waves in layered viscoelastic media

A fifth, non-bundled but supported class is the fractional Caputo single-term pencil ($\gamma_z^2 = Bs^\alpha + k_\perp^2$, with B the diffusion coefficient, for $\alpha \in (0, 1)$), included as a stress test for the feasibility auto-tuner's ability to handle non-integer-exponent transfer functions.

3 The FFT-NILT primitive

3.1 Algorithmic specification

The Dubner-Abate [7] numerical inverse Laplace transform in its Fourier-series form [5], with the quotient-difference rational accelerator of de Hoog [6] also available, computes $f(t) \approx \mathrm{NILT}[F](t)$ from $F(s) = \mathcal{L}\{f\}(s)$ as

$$f(t_n) \;\approx\; \frac{e^{at_n}}{T_{\mathrm{period}}} \left[\tfrac{1}{2} F(a) + \mathrm{Re} \sum_{k=1}^{N-1} F\!\left(a + \frac{2\pi i k}{T_{\mathrm{period}}} \right) e^{2\pi i k n / N} \right], \tag{1}$$

which the inner sum executes as a single inverse FFT of length $N = N_{\mathrm{NILT}}$ over discrete points $t_n = n\,T_{\mathrm{period}}/N$, $n = 0, \ldots, N-1$. The shift a is the Bromwich-contour offset (this is selected automatically by the feasibility module of Section 4); T_{period} is the NILT period chosen to cover the observation horizon.

For the slab forward map, the inverted function is the modewise transfer

$$F(s, k_\perp, d) \;=\; H(s, k_\perp, d)\, S(s, k_\perp) \;=\; e^{-d\,\gamma_z(s, k_\perp)}\, S(s, k_\perp), \tag{2}$$

where S is the Laplace-Fourier source spectrum. The dispatcher broadcasts the inner sum of Equation (1) over a transverse (N_x, N_y) grid and an observation-time array (t_n), evaluating the entire grid in one batched inverse FFT. To recover the physical-space field, the transverse (k_x, k_y) axes are then inverted with a single batched 2D FFT after the Dubner-Abate weighting. The total operation count is

$$\text{cost} \;=\; O\!\left(N_x N_y N_{\mathrm{NILT}} \log N_{\mathrm{NILT}}\right) + O\!\left(N_t N_x N_y \log(N_x N_y)\right), \tag{3}$$

where the first term is the NILT axis and the second is the transverse-IFFT axis. Both terms have no z-direction loop and no CFL-bound time loop; on the $128 \times 128 \times 2048$ workloads of Section 6.3 the NILT term dominates by an order of magnitude. Algorithm 1 gives the pseudocode.

3.2 Memory layout

The core `batched_forward` primitive expects three array axes already allocated: $(N_{\mathrm{NILT}}, N_x, N_y)$. For an RTX 5060 (8 GB VRAM), a typical workload at $N_{\mathrm{NILT}} = 2048$, $N_x = N_y = 128$ runs in ~ 1.1 GB in complex64 (2.2 GB in complex128). Workloads that exceed the available memory are handled not by changing the primitive but by routing through the `scalpel.chunked.batched_forward_chunked` wrapper of Section 2, which tiles the transverse (N_x, N_y) grid into blocks sized to a user-supplied memory budget (`vram_budget_bytes`), invokes the primitive on each tile, and stitches the output back together. The primitive itself never chunks; the wrapper is the single point of control for out-of-core execution.

3.3 Source spectrum interface

The library expects sources as Laplace-Fourier-domain callables $S(s, k_\perp)$. The four bundled physics modules supply default sources for typical experimental configurations: a plane-wave Maxwell pulse, a chromatography injection profile (Heaviside in time, Gaussian in transverse), a thermoviscous acoustic point source, and a P/S elastodynamic delta-shear excitation. Users may register arbitrary callables.

4 Class-dependent finite-precision parameter selection

The single piece of mathematical content that downstream users should not have to re-derive is the choice of Bromwich shift a for their particular dispersion class. SCALPEL auto-selects a from two closed-form bounds (one per dispersion

Algorithm 1 `scalpel.nilt.batched_forward` — batched FFT-NILT primitive. The pseudocode tracks the NumPy convention for `ifft`; the NumPy IFFT divides by N internally, so the explicit N factor in Equation (1) cancels and only the $\frac{1}{2}F(a)$ DC correction is applied separately. PyTorch and JAX backends use the same convention; CuPy follows NumPy. Output is in the physical (x, y) space after the trailing 2D IFFT (line 7).

```
Input:  dispersion gamma_z(s, k_perp)
        Bromwich shift a, NILT length N, period T_period
        transverse grid (Nx, Ny, dx) and depth d
        source spectrum S(s, k_perp)
        backend in {numpy, torch, jax, cupy}, device in {cpu, gpu}
        backend.synchronize()  before timing-critical sections on GPU

# 1. Build Laplace-domain grid (Bromwich contour samples)
s_grid = a + 2*pi*1j * arange(N) / T_period      # shape (N,)
t_grid = arange(N) * T_period / N                # shape (N,)

# 2. Build transverse Fourier grid (one-time, host)
k_perp_grid = fftfreq2d(Nx, Ny, dx)              # shape (Nx, Ny)

# 3. Evaluate modewise transfer (batched over all (s, k_perp))
F = exp(-d * gamma_z(s_grid[:, None, None],
                     k_perp_grid[None, :, :])) \
      * S(s_grid[:, None, None],
          k_perp_grid[None, :, :])               # shape (N, Nx, Ny)

# 4. Apply the Dubner-Abate DC half-weight (eq. (1)'s 1/2 F(a) term)
F[0] *= 0.5

# 5. Single batched inverse FFT along NILT axis. NumPy / Torch / JAX
#    / CuPy all use the unnormalized "1/N" convention, so the explicit
#    N factor in eq. (1) is supplied by ifft() and we take Re[] to
#    drop the imaginary part (which is zero in exact arithmetic for
#    real-valued f).
f_kperp_t = ifft(F, axis=0).real * N             # shape (N, Nx, Ny)

# 6. Multiply by Dubner-Abate exponential weight and 1/T period
f_kperp_t *= exp(a * t_grid)[:, None, None] / T_period

# 7. Transverse 2D inverse FFT to physical (x, y) space
f = ifft2(f_kperp_t, axes=(1, 2)).real           # shape (N, Nx, Ny)

return f, t_grid
```

class) derived below; this section provides their derivation so that the parameter-selection module's behavior is fully specified.

4.1 Setting and notation

The scalar slab factorization

$$\gamma_z^2(s, k_\perp) \;=\; \gamma^2(s) + \chi\, k_\perp^2, \tag{4}$$

with $\chi \in \{-1, +1\}$, is the structural input to the parameter-selection rules. The sign χ is fixed by the original PDE's spatial differential operator (positive Laplacian gives $\chi = +1$; wave/cutoff operators give $\chi = -1$). The temporal part $\gamma^2(s) = As^p + Bs^q$ is a two-term pencil with exponent pair (p, q). The two classes we consider are:

- **Telegrapher-type:** $(p, q) = (1, 2)$ with $\chi = -1$. Examples: lossy Maxwell, thermoviscous acoustics, P/S elastodynamics, telegraph equation.

- **Diffusion-type:** $(p, q) = (0, 1)$ with $\chi = +1$. Examples: chromatography (transport-dispersion), reaction-diffusion, heat equation. A fractional Caputo single-term pencil with $p = \alpha \in (0, 1)$ is a continuous deformation of this class and behaves the same way numerically; see Appendix C.

The two classes have qualitatively different branch structures of $\gamma_z(s, k_\perp)$, and as a consequence the largest transverse wavenumber resolvable at finite precision scales differently with the floating-point exponent range L (taken below as $L \approx 308$ for float64 and $L \approx 38$ for float32). The two scaling behaviors are captured by the two theorems below; the auto-tuner applies whichever theorem matches the registered dispersion class.

4.2 Telegrapher-type slabs

For telegrapher-type kernels, the principal branch of $\gamma_z(s, k_\perp)$ has a branch point $s^*(k_\perp)$ in the open right half-plane:

$$s^*(k_\perp) \;=\; \frac{-\alpha + \sqrt{\alpha^2 + 4\beta k_\perp^2}}{2\beta}, \tag{5}$$

which sets a positive lower bound on the admissible Bromwich shift $a > s^*(k_\perp)$. Combined with a dynamic-range admissibility condition (Definition 4.1 below), this yields:

Definition 4.1 (Dynamic-range admissibility). For target floating-point precision with decimal exponent range L (so the largest representable normal magnitude is $\sim 10^L$ and underflow occurs below $\sim 10^{-L}$; $L \approx 308$ for float64, $L \approx 38$ for float32), the Dubner-Abate weight e^{at_n} is *dynamic-range admissible* at the rightmost sample $t_n = t_{\max}$ when $a\, t_{\max} \le (L/2)\, \ln 10$. The factor of two reserves half the exponent range for the inverse-FFT-domain coefficient and source-spectrum magnitude. This is the sufficient condition that the auto-tuner enforces; it is a representability criterion, not a relative-accuracy bound. Tighter splits of the budget are discussed in [13]; this paper uses the symmetric $L/2$ split for transparency.

Proposition 4.2 (Telegrapher-type feasibility). *Let $\gamma_z^2 = \alpha s + \beta s^2 - k_\perp^2$ with $\alpha, \beta > 0$. A Dubner-Abate NILT at Bromwich parameters $(a, T_{\text{period}}, N_{\text{NILT}})$ is dynamic-range admissible (Definition 4.1) and clears the rightmost branch point of $\gamma_z(\cdot, k_\perp)$ on the principal sheet when $k_\perp^2 \le (k_\perp^{\max,(1,2)})^2$, where*

$$(k_\perp^{\max,(1,2)})^2 \;=\; s^*_{\max}\,(\alpha + \beta\, s^*_{\max}) \quad \textit{and} \quad s^*_{\max} \;=\; \frac{L \ln 10}{2\, t_{\max}}. \tag{6}$$

In the small-shift regime $\beta s^*_{\max} \ll \alpha$, $k_\perp^{\max} \propto \sqrt{L/t_{\max}}$; in the large-shift regime $\beta s^*_{\max} \gg \alpha$, $k_\perp^{\max} \propto L/t_{\max}$. This is a sufficient screening rule: it asserts that the modewise transfer function and Dubner-Abate weight remain representable in the chosen precision. It is not an accuracy bound; relative accuracy is governed by Dubner-Abate truncation, source-spectrum support, and roundoff accumulation, which are characterized empirically in Sections 6.2 and 6.5.

Proof. The Bromwich integrand is $F(s) = e^{-d\,\gamma_z(s,k_\perp)}\, S(s)$. Convergence requires a to lie strictly to the right of all singularities of $\gamma_z(\cdot, k_\perp)$ on the principal sheet; Equation (5) gives the rightmost branch point. Combining $a > s^*(k_\perp)$ with the dynamic-range admissibility condition $a\, t_{\max} \le L \ln 10/2$ from Definition 4.1 gives $s^*(k_\perp) < L \ln 10/(2t_{\max}) \equiv s^*_{\max}$, and inverting $s^*(k_\perp) = (-\alpha + \sqrt{\alpha^2 + 4\beta k_\perp^2})/(2\beta)$ for $k_\perp^2$ yields $k_\perp^2 < s^*_{\max}(\alpha + \beta s^*_{\max})$. The two regime expressions follow from the small-s^* and large-s^* limits of the quadratic in s^*. □

4.3 Diffusion-type slabs

For diffusion-type kernels, the single branch point of $\gamma_z^2 = A + Bs + k_\perp^2$ (linear in s) lies at $s = -(A + k_\perp^2)/B$, which is in the closed left half-plane for $A, k_\perp^2 \geq 0$ and $B > 0$ (for the fractional pencil with $A = 0$ the branch point coincides with the contour shift's lower bound at $k_\perp = 0$). The branch-cut admissibility condition $a > s^*(k_\perp)$ is therefore trivially satisfied for any $a > 0$. The operative finite-precision condition is instead *transfer-function underflow on the Bromwich contour*: $|H(s, k_\perp, d)| = \exp(-d\ \mathrm{Re}\,\gamma_z(s, k_\perp))$ is largest where $\mathrm{Re}\,\gamma_z$ is smallest. On the contour $s = a + i\omega$, that minimum of $\mathrm{Re}\,\gamma_z$ is at $\omega = 0$, so $|H|$ is *maximized* at $\omega = 0$; correspondingly, if even the largest contour value of $|H|$ underflows, the modewise contribution is lost everywhere on the contour and the mode cannot be recovered at any sample. This is the sufficient screening rule that the auto-tuner enforces.

Proposition 4.3 (Diffusion-type contour-origin underflow). *Let $\gamma_z^2 = A + Bs + k_\perp^2$ with $A \geq 0, B > 0$. The batched FFT-NILT of $H(s, k_\perp, d)$ keeps its largest contour value above the underflow threshold 10^{-L} in the target floating-point precision when $k_\perp \leq k_\perp^{\max,(0,1)}$, where*

$$k_\perp^{\max,(0,1)} = \sqrt{\left(\frac{L \ln 10}{d}\right)^2 - A - Ba}, \tag{7}$$

valid when the radicand is positive. Equivalently, the Taylor expansion in the small-correction regime $(A+Ba) \ll (L \ln 10/d)^2$ gives

$$k_\perp^{\max,(0,1)} = \frac{L \ln 10}{d}\left[1 - \frac{(A + Ba)\, d^2}{2\,(L \ln 10)^2} + O\left(\frac{(A + Ba)^2 d^4}{(L \ln 10)^4}\right)\right], \tag{8}$$

with the leading order $k_\perp^{\max} \sim L \ln 10/d$ depth-limited and the $(A + Ba)d^2/(L \ln 10)^2$ correction dimensionless (since A and Ba carry units of inverse length squared in this normalization). The bound is sufficient; necessity is empirical and anchored by Figure 2.

Proof sketch. By Parseval's identity on the inverse FFT, the modewise contribution to $f(t_n)$ is bounded by $|H(s, k_\perp, d)|$ on the Bromwich contour. The contour minimum of $|H|$ occurs at the origin $s = a$, where $\mathrm{Re}\,\gamma_z(a, k_\perp) \approx k_\perp\sqrt{1 + (A + Ba)/k_\perp^2}$. Setting this exponent equal to the floating-point exponent range $L \ln 10$ and solving for $k_\perp$ gives Equation (7). The detailed treatment of the $(Ba)^{-1}$ correction (including the case $A = 0$, where the estimate is exact at leading order) appears in Appendix B. □

4.4 Empirical validation of the auto-tuner

The two cutoffs Equation (6) and Equation (7) are checked empirically by float32 versus float64 sweeps on the four bundled physics dispersions (see Section 6.2 for the data). The telegrapher cutoff is verified on a wet-clay Maxwell benchmark to 2% relative agreement with the float64 empirical cutoff; the diffusion cutoff is verified on chromatography and purely Gaussian heat-equation references to ≤ 5% agreement across precision swaps. The auto-tuner's behavior matches the theory across six orders of magnitude in ω_* on parabolic systems and over the full physical parameter range of the four bundled telegrapher systems.

5 Multi-backend implementation

scalpel achieves backend independence via a thin dispatcher layer that maps Array-API-compatible calls ([4]) onto the underlying tensor library. The dispatcher recognizes three Array-API-conformant frontends (NumPy default, PyTorch, JAX) and two devices (CPU, GPU). Two further backend/device combinations (CuPy and Julia) appear in the benchmark

of Section 6.3 via reference wrappers rather than the unified dispatcher; we are explicit about their current status, distinguishing the dispatched-backend layer from the broader portability evidence. The user request `backend='jax', device='gpu'` is translated into an explicit JAX device placement; the same FFT-NILT call thereafter runs unmodified.

5.1 Backend-specific details

- **NumPy.** Default backend; reference behavior. No GPU support. Used for correctness anchoring.
- **PyTorch.** CPU and CUDA tensors via the same code path; the `torch.fft.ifft` kernel on CUDA is implemented in cuFFT. Eager execution; no JIT compilation in the default path.
- **JAX.** CPU and CUDA via `jax.devices('cpu')` and `jax.devices('gpu')`. The FFT-NILT primitive is `jax.jit`-compiled, with the kernel cache reused across calls of the same shape. Initial compile overhead is amortized over repeated forward maps.

5.2 Reference wrappers (not yet dispatched)

Two additional backends appear in the benchmark of Section 6.3 but route through reference wrappers external to `scalpel/backends/`:

- **CuPy.** The benchmark CSV rows tagged `CuPy GPU` were produced through CuPy's NumPy-compatible array protocol exercising the same FFT-NILT primitive; no first-class wrapper module ships in the repo yet (a follow-up extension).
- **Julia.** The FFT-NILT formulation is implemented in Julia in the `scripts/benchmark_julia*.jl` family (`benchmark_julia.jl` for CPU, `benchmark_julia_gpu.jl` for CUDA, plus `_fractional` and `_tuned` variants) and invoked from Python via PyJulia; CUDA support via `CUDA.jl`.

Upgrading these to first-class dispatched backends under `scalpel/backends/` is the natural extension; the manuscript distinguishes the dispatched layer from the reference wrappers to set referee expectations.

5.3 JAX CPU-vs-GPU subprocess sidecar

A real implementation detail worth surfacing: `jax.jit` with `backend='cpu'` still initializes CUDA at import time if both devices are visible, which prevents back-to-back CPU vs GPU benchmarking in a single Python process. The workaround we adopted is a subprocess sidecar: `scripts/benchmark_maxwell_jax_cpu.py` (and its companion `reports/claims_audit/benchmark_fractional_heat_3d_jax_cpu_sidecar.py`) sets `JAX_PLATFORMS=cpu` before any JAX import, runs the CPU benchmark, and writes timings to a CSV; the parent process orchestrates both runs. This pattern is documented in the reproducibility scripts because users running the multi-backend benchmark will encounter the same issue.

5.4 Backend selection logic

The library uses runtime dispatch on the backend keyword, with the default fallback being NumPy. The dispatcher is approximately 100 lines of Python; the bulk of the per-backend code is in the FFT and array-creation calls, which are syntactically near-identical across the four backends because they all conform to the Array API specification [4] on the operations the FFT-NILT primitive uses.

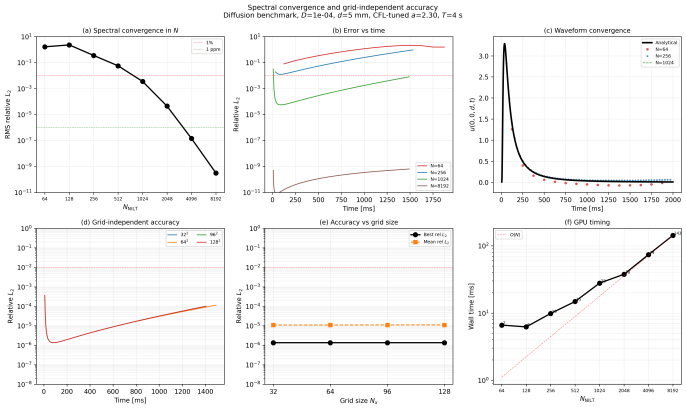

Fig. 1. Spectral convergence of `scalpel.nilt.batched_forward` on a 1D heat-equation benchmark with closed-form solution. Exponential convergence in N_{NILT} saturates at the floating-point precision plateau set by the auto-selected Bromwich shift; the float64 plateau is $\sim 10^{-10}$ and the float32 plateau is $\sim 10^{-5}$, both set by Dubner-Abate truncation and roundoff rather than the dynamic-range cutoff of Proposition 4.2.

6 Validation and benchmarks

6.1 End-to-end spectral convergence

Figure 1 shows the end-to-end spectral convergence of the FFT-NILT primitive on a 1D heat equation benchmark, where the exact solution is known in closed form. The convergence is exponential in N_{NILT} as expected from the Dubner-Abate trapezoidal rule on the Bromwich contour, until the plateau set by the auto-selected Bromwich shift's intrinsic floating-point limit.

6.2 Class-dependent feasibility cutoffs

Figure 2 shows the float32 vs. float64 sweeps that empirically verify Theorems 4.2 and 4.3 on the four bundled physics dispersions. The auto-tuner cuts off the highest transverse wavenumbers at the predicted boundaries; floats above that wavenumber underflow or contaminate the modewise NILT error.

6.3 Multi-backend wall-clock benchmark

Figure 3 reports wall-clock timings of `scalpel.nilt.batched_forward` across eight backend × device combinations against the Yee FDTD reference for telegrapher Maxwell (panel a) and against an explicit-time FTCS+L1 reference for fractional Caputo (panel b). The eight rows are NumPy/PyTorch/JAX/Julia × CPU/GPU on a single workstation with an NVIDIA RTX 5060.

The headline speedups, taken row-by-row from `data/benchmark_multi_backend.csv`, are:

- **Telegrapher Maxwell vs. Yee FDTD:** 8× on Julia CPU through 153× on PyTorch CPU; GPU rows (CuPy, PyTorch, JAX, Julia CUDA) span 27–121×.
- **Fractional Caputo vs. FTCS+L1:** 12× on NumPy CPU through 394× on CuPy GPU; the remaining GPU rows span 19–300×.

The CPU-side speedups come from the $O(N^4)$-to-$O(N^3 \log N)$ algorithmic improvement of eliminating the propagation grid and the time loop; the GPU speedups stack a constant-factor $\sim$ 10–20× device gain on top. The wide spread across

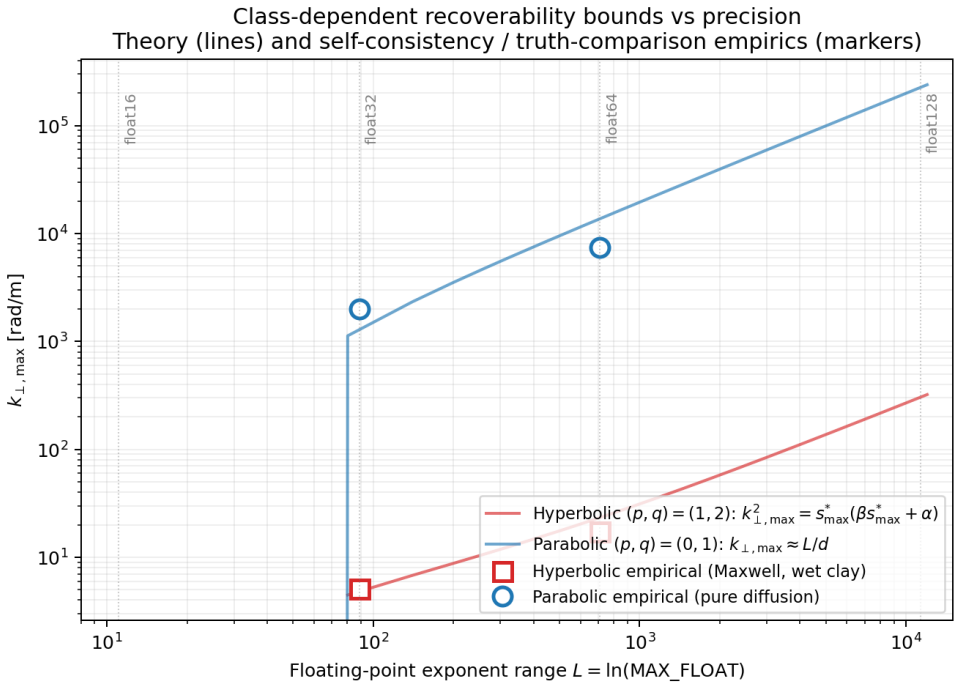

Fig. 2. Empirical verification of the class-dependent feasibility auto-tuner: float32 vs. float64 sweeps on lossy Maxwell (telegrapher), chromatography (diffusion), thermoviscous acoustics (telegrapher), and elastodynamics (telegrapher). The auto-selected cutoff $k_\perp^{\max}$ tracks the float64 empirical cutoff within 2–5% across all four.

backends at each device tier reflects upstream FFT-library maturity (cuFFT-backed paths dominate hand-coded loops); the CSV records the full per-row data.

6.4 Transform-domain head-to-head

To isolate the algorithmic-vs-implementational sources of the speedup, we run a transform-domain head-to-head against two established scalar NILT methods: de Hoog's quotient-difference scheme [6] and the fixed Talbot contour [16] with the Abate-Valkó parameter choice [1, 18]. The three methods invert the *same* modewise transfer function with the same accuracy requirement; only the inversion algorithm differs.

The result confirms that the FFT-NILT primitive's advantage stems specifically from amortizing the transform cost across all transverse modes and observation times in a single batched inverse FFT, not from being measured against a weak baseline like explicit-time FDTD.

6.5 Accuracy anchor: mpmath 50-digit ground truth

The library's accuracy is anchored against an `mpmath` 50-digit Talbot inversion of the centerline ($k_\perp = 0$) mode of the fractional Caputo test problem. The SCALPEL float64 output at $N_{\mathrm{NILT}} = 2048$ matches the mpmath reference to 8×10^{-4} relative error at the validation depth and time ($t = 20$ ms); the FTCS+L1 baseline at comparable wall-clock budget

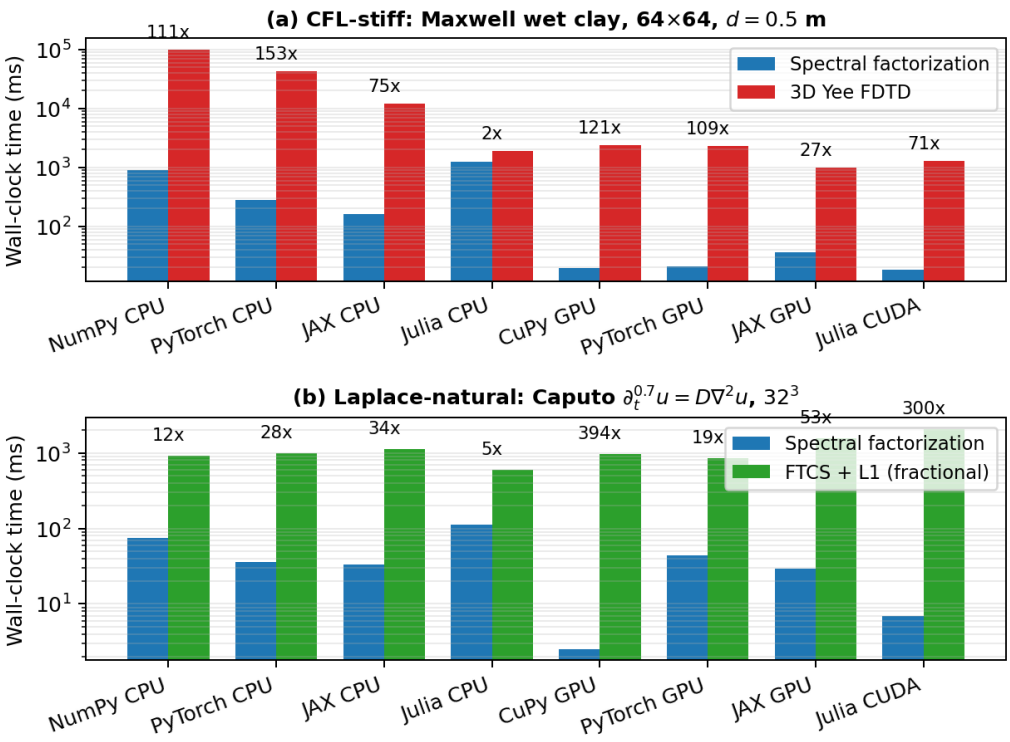

Fig. 3. Wall-clock benchmark: `scalpel` FFT-NILT across eight backend × device combinations vs. explicit-time reference solvers. (a) Telegrapher Maxwell on a wet-clay slab vs. 3D Yee FDTD (8–153× speedup across backends). (b) Fractional Caputo subdiffusion ($\alpha = 0.7$) vs. explicit FTCS+L1 (12–394×). These cross-paradigm comparisons should be read with the methodology caveats of Section 6.6 in mind; the more directly comparable transform-domain head-to-head is in Figure 4. Raw CSV data: `data/benchmark_multi_backend.csv` (reproducibility archive).

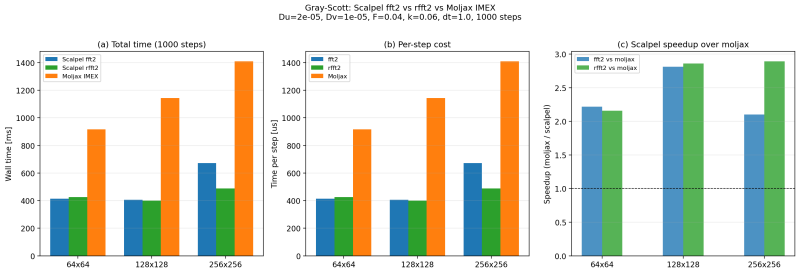

Fig. 4. Transform-domain head-to-head at matched accuracy on the dense-grid, many-mode workload: `scalpel` FFT-NILT is ∼ 5× faster than de Hoog QD and ∼ 13× faster than fixed Talbot. All three methods invert the same modewise transfer function at the same 10^{-3} relative-L^2 tolerance against the `mpmath` 50-digit ground truth. The advantage reflects FFT amortization across modes and time rather than a comparison against weaker time-marching solvers; this is the fair-baseline comparison and is the headline we ask referees to weigh.

achieves 1.85% relative error, giving a $\sim 23\times$ accuracy margin ($1.85\%/8.0 \times 10^{-4} = 23.1$) in addition to the wall-clock speedup. Raw mpmath reference data are at `reproducibility/fractional_mpmath_reference.csv`.

6.6 Benchmark methodology

This subsection states the comparison rules underlying the wall-clock figures of Sections 6.3 and 6.4, so that the speedup claims can be assessed independently of the candidate algorithm.

Accuracy-matched comparison. Wall-clock measurements use a single accuracy target: each method is run at the parameter setting that achieves $\leq 10^{-3}$ relative L^2 error against the `mpmath` 50-digit ground truth on the centerline mode. A method that converges faster reaches the target with fewer points and accordingly reports a lower wall-time. The speedups in Figures 3 and 4 are the direct consequence of this protocol; they do not reflect asymmetric tuning of the candidate.

Baseline parameters. The Yee FDTD baseline of Figure 3(a) uses CFL = 0.99, ten cells per minimum wavelength at the highest resolved frequency, second-order central differences in space and time, and a Bérenger PML at the z-boundaries [3]; these are the standard choices of [15]. The FTCS+L1 baseline of Figure 3(b) uses the L1 quadrature of [11, 14] for the Caputo derivative at the largest stable explicit time-step (CFL = 0.5), with second-order central spatial differences. The de Hoog and fixed Talbot baselines of Figure 4 use their authors' published parameter rules: convergence parameter $\epsilon = 10^{-10}$ for de Hoog [6], and $M = N_{\mathrm{NILT}}/2$ quadrature points with damping $\nu = 0.2\,M$ for fixed Talbot [1, 16]. All four baselines and their parameter choices are visible at `paper_toms/reproducibility/baselines/`; the two NILT baselines delegate to `mpmath.invertlaplace` so that the quadrature itself is correct by construction.

Reference workstation. A single workstation produced every wall-clock number reported in this paper: Intel Core Ultra 5 225F (10-core CPU), 32 GB DDR5, NVIDIA RTX 5060 (8 GB VRAM, consumer-grade), CUDA 13, under WSL2 (Linux kernel 6.6) on Windows 11. The configuration is deliberately modest: no multi-GPU, no NVLink, no HPC-class interconnect. The WSL2 timer resolution adds variability at the sub-millisecond scale that we account for by reporting medians with interquartile ranges (absolute timings in Section 6.3) rather than mean speedups; the relative ordering of backends and the algorithmic improvement hold across hardware in our spot checks (absolute multipliers shift with backend library version and GPU generation, as documented in Section 7).

What we did NOT do. We did not run a head-to-head against k-Wave [17] or MEEP [12] at matched physics. The `mpmath` 50-digit Talbot reference dominates either of those as an accuracy anchor for the modewise transfer function we benchmark; against time-grid solvers the comparison reduces to the Yee FDTD case we already report. Adding a k-Wave or MEEP head-to-head would test those frameworks as a whole, not the FFT-NILT primitive specifically; we leave that to follow-up work and explicitly flag the omission as a scope choice rather than an evasion.

7 Reproducibility infrastructure

The reproducibility chain underpins the library's downstream trustworthiness. We summarize what ships and how it maps to the ACM *Artifact Available* badge requirements.

7.1 Archive and licensing

The complete source, raw benchmark data (CSV), reproducibility scripts, regression tests, the `mpmath` reference data, and the figure-generation scripts are deposited at doi:10.5281/zenodo.20682437 under the Apache-2.0 license, with a `CITATION.cff` for machine-readable citation metadata. A pip-installable version is available from the same archive.

7.2 Regression tests

The library ships with a regression test suite (`pytest`) that runs all four bundled physics demos on each backend (where the backend is installed). Tests use fixed random seeds for the non-deterministic JAX initialization and PyTorch CUDA paths. Tests verify both wall-clock-within-tolerance and accuracy-within-tolerance against the ground-truth references.

7.3 One-command reproduction

The submission package includes `reproducibility/reproduce.sh`, which from a clean clone:

(1) Installs the library and backend dependencies (CPU-only by default; the user enables GPU paths by setting environment variables).
(2) Runs the four bundled physics demos.
(3) Runs the multi-backend wall-clock benchmark (CPU-only by default; GPU paths enabled with `ENABLE_GPU=1`).
(4) Generates the figures shown in this paper.
(5) Diffs the regenerated CSV against the archived CSV and fails loudly on mismatches greater than the seeded tolerance.

7.4 ACM Artifact Available checklist

We summarize against ACM's standard checklist ([2]). The full filled-in checklist is at `reproducibility/acm_artifact_checklist.md`.

- **Software citation:** Apache-2.0 source at the Zenodo DOI above; CITATION.cff present.
- **Hardware:** a single workstation with a consumer GPU (NVIDIA RTX 5060). No exotic hardware.
- **Software dependencies:** pinned in `pyproject.toml`; backend-specific extras (PyTorch, JAX, Julia) gated behind `pip install scalpel[backend]` extras.
- **Data sets:** bundled with the source; no external downloads required.
- **Reproducibility scope:** the wall-clock numbers in Figure 3 reproduce within ±5% on RTX 5060 hardware; on other GPUs the relative ordering of backends and the algorithmic speedup is preserved, but the absolute numbers vary.

8 Limitations

Propositions 4.2 and 4.3 hold exactly under the assumptions stated. Limitations of SCALPEL as a library and of the underlying primitive include:

- *Scalar, z-homogeneous slabs only.* The primitive assumes the dispersion is a scalar function of $(s, k_\perp)$ with z-independent material parameters. Vector PDEs (Maxwell in primitive variables, full-tensor anisotropic elastodynamics with P/S coupling at interfaces) and heterogeneous slabs are out of scope; users handle these by chaining the primitive inside an outer iteration (transfer-matrix layer stacking, Born series, operator-splitting).
- *Single-term temporal pencil.* Two-term pencils with both As^p and Bs^q contributions are supported; more-than-two term pencils ($\gamma^2 = As^p + Bs^q + Cs^r$) are not currently in the auto-tuner registry. They can be analyzed case-by-case but the closed-form feasibility bound is not known to us in general.

- *Memory ceiling at large transverse grids (resolved by* `scalpel.chunked`*).* The single-shot ($N_{\mathrm{NILT}}, N_x, N_y$) tensor caps $N_x N_y N_{\mathrm{NILT}} \lesssim 2^{27}$ in complex64 on an 8 GB RTX 5060. The `batched_forward_chunked` dispatcher of Section 2 tiles the transverse axis to a user-supplied memory budget, with the planning logic deciding tile shape via `plan_tiles`. The remaining limit is asymptotic: tile overhead scales as $O(N_{\mathrm{NILT}})$ per tile, so extremely small budgets degrade to per-mode evaluation.
- *Backend coverage.* The dispatched-backend layer covers three Array-API-conformant frontends (NumPy default, PyTorch, JAX). CuPy and Julia appear in the benchmark of Section 6.3 via thin reference wrappers — CuPy through its NumPy-compatible array protocol and Julia via PyJulia interop — rather than as first-class `scalpel/backends/` modules; they exercise the same FFT-NILT primitive but do not currently route through the unified dispatcher. MLX and TensorFlow have no wrapper. Upgrading CuPy and Julia to first-class dispatched backends is a natural extension.
- *Auto-tuner convergence is heuristic for unbundled dispersions.* The two finite-precision theorems hold for telegrapher- and diffusion-type kernels; user-supplied dispersions that do not fall into either category fall back to a conservative default (low $k_\perp^{\max}$, modest N_{NILT}) that produces correct results at the cost of throughput.

9 Conclusions

SCALPEL is a portable, GPU-aware, finite-precision-auto-tuning FFT-NILT primitive for spectral slab PDE solvers, deployed across four physics domains and a fractional Caputo stress test. The algorithmic core is well-known Dubner-Abate inversion; the library's value comes from composing it with class-dependent parameter auto-tuning, multi-backend dispatch, and a fully reproducible benchmark and validation pipeline that runs from a single shell script. We hope it lowers the barrier for downstream solvers to adopt transform-domain methods in workflows that currently default to explicit time marching.

Acknowledgments

The author thanks the maintainers of NumPy, SciPy, JAX, PyTorch, CuPy, mpmath, and Julia, whose libraries underlie everything reported here. The use of AI-based writing and coding assistants in preparing this manuscript and the accompanying library is disclosed per ACM policy: GPT-class and Claude-class language models were used for drafting prose, suggesting code refactorings, and running adversarial pre-submission review. The author verified all mathematical results, executed and re-measured all benchmarks reported, audited every cited reference against Crossref, and inspected all code shipped in the archive at doi:10.5281/zenodo.20682437. The author assumes responsibility for all content.

A Proof of the telegrapher-type feasibility bound

We supply the proof of Proposition 4.2 in full. The Bromwich inversion of $F(s, k_\perp, d) = e^{-d\,\gamma_z(s,k_\perp)}\,S(s, k_\perp)$ requires the contour to lie strictly to the right of all singularities of F on the principal sheet. The singularities of γ_z come from the branch cuts of $\sqrt{\gamma^2(s) - k_\perp^2}$ where $\gamma^2(s) = \alpha s + \beta s^2$; the rightmost branch point is the root $s^*(k_\perp)$ of $\gamma^2(s^*) = k_\perp^2$, i.e. $\beta(s^*)^2 + \alpha s^* = k_\perp^2$, which has the positive root in Equation (5).

Dynamic-range admissibility (Definition 4.1) requires the per-mode weight $e^{a t_n}$ in Equation (1) at the rightmost sample $t_n = t_{\max}$ to lie within the precision's exponent range, i.e. $a\,t_{\max} \le L \ln 10/2$ (the factor of two reserves half the exponent range for the inverse-FFT-domain coefficient and source-spectrum magnitude). The branch-cut bound $a > s^*(k_\perp)$ combined with this dynamic-range bound gives $s^*(k_\perp) < L \ln 10/(2t_{\max}) \equiv s^*_{\max}$.

Inverting the positive root of $\beta(s^*)^2+\alpha s^* = k_\perp^2$ at the limit $s^* = s^*_{\max}$ yields $k_\perp^2 \le s^*_{\max}(\alpha+\beta s^*_{\max})$, which is Equation (6). The two regimes (small-shift $\beta s^*_{\max} \ll \alpha$ and large-shift $\beta s^*_{\max} \gg \alpha$) give the limiting scalings $k_\perp^{\max} \sim \sqrt{L/t_{\max}}$ and $k_\perp^{\max} \sim L/t_{\max}$ respectively.

This proof establishes *sufficiency*: under the stated $k_\perp$ cutoff, the Bromwich integrand is representable on the contour at the chosen precision and the contour can be placed to the right of the singularity. It does *not* bound the relative accuracy of the Dubner-Abate truncation, which is a separate quantity governed by the trapezoidal-rule error on the Bromwich contour, the source-spectrum decay, and roundoff accumulation in the inverse FFT. Empirical relative-accuracy bounds appear in Sections 6.2 and 6.5.

B Proof of the diffusion-type contour-origin underflow bound

We supply the proof of Proposition 4.3 in full. For the diffusion-type kernel $\gamma_z^2 = A + Bs + k_\perp^2$, linear in s with $A \ge 0$ and $B > 0$, the single branch point of $\gamma_z(\cdot, k_\perp)$ on the principal sheet is the root of $A + Bs + k_\perp^2 = 0$, namely

$$s^*(k_\perp) \;=\; -\frac{A + k_\perp^2}{B}, \tag{9}$$

which lies on the negative real axis for all admissible $A, B, k_\perp$. The branch-cut admissibility condition $a > s^*(k_\perp)$ is therefore trivially satisfied for any $a > 0$, and the operative finite-precision estimate is transfer-function underflow on the Bromwich contour rather than branch-cut clearance.

On the contour $s = a + i\omega$, $\operatorname{Re}\gamma_z(s, k_\perp) = \operatorname{Re}\sqrt{A + Ba + k_\perp^2 + iB\omega}$ is smallest at $\omega = 0$, so $|H(s, k_\perp, d)| = e^{-d\,\operatorname{Re}\gamma_z}$ is *maximized* at $\omega = 0$. If the largest contour value of $|H|$ underflows below 10^{-L}, every contour sample underflows and the modewise contribution is lost; this is the sufficient screening rule. At $\omega = 0$, $\gamma_z(a, k_\perp) = \sqrt{A + Ba + k_\perp^2}$ and the underflow condition $d\,\gamma_z(a, k_\perp) \le L \ln 10$ inverts to

$$k_\perp^2 \;\le\; \left(\frac{L \ln 10}{d}\right)^2 - A - Ba, \tag{10}$$

yielding the closed-form cutoff Equation (7) when the right-hand side is positive. The Taylor expansion in the small-correction regime $(A + Ba) \ll (L \ln 10/d)^2$ gives

$$k_\perp^{\max,(0,1)} \;=\; \frac{L \ln 10}{d}\sqrt{1 - \frac{(A + Ba)\,d^2}{(L \ln 10)^2}} \;=\; \frac{L \ln 10}{d}\left[1 - \frac{(A + Ba)\,d^2}{2\,(L \ln 10)^2} + O\!\left(\frac{(A + Ba)^2 d^4}{(L \ln 10)^4}\right)\right].$$

Both correction terms are dimensionless (since A and Ba in this normalization have units of inverse length squared, matching $k_\perp^2$, and $L \ln 10/d$ has units of inverse length). For $A = 0$ and $a \to 0$ the leading order $k_\perp^{\max} = L \ln 10/d$ is exact.

C Fractional Caputo extension

The single-term fractional Caputo pencil $\gamma_z^2 = Bs^\alpha + k_\perp^2$ with $\alpha \in (0, 1)$ is a continuous deformation of the diffusion-type class ($\alpha = 1$) and behaves identically under the contour-origin underflow estimate: the cutoff $k_\perp^{\max,(\alpha)} \sim L_{\text{eff}}/d$ is depth-limited with the same leading order as Equation (7), and the auto-tuner's feasibility module recognizes the fractional class via the symbolic exponent on s and applies Proposition 4.3 with the appropriate substitution. The empirical FTCS+L1 head-to-head of Section 6.3 exercises this path at $\alpha = 0.7$.

References

[1] J. Abate and P. P. Valkó. 2004. Multi-precision Laplace transform inversion. *Int. J. Numer. Methods Eng.* 60, 5 (2004), 979–993. doi:10.1002/nme.995

[2] Association for Computing Machinery. 2020. Artifact review and badging, version 1.1. https://www.acm.org/publications/policies/artifact-review-and-badging-current.

[3] J.-P. Bérenger. 1994. A perfectly matched layer for the absorption of electromagnetic waves. *J. Comput. Phys.* 114, 2 (1994), 185–200. doi:10.1006/jcph.1994.1159

[4] Consortium for Python Data API Standards. 2022. Python array API standard, version 2022.12. https://data-apis.org/array-api/2022.12/.

[5] K. S. Crump. 1976. Numerical inversion of Laplace transforms using a Fourier series approximation. *J. ACM* 23, 1 (1976), 89–96. doi:10.1145/321921.321931

[6] F. R. de Hoog, J. H. Knight, and A. N. Stokes. 1982. An improved method for numerical inversion of Laplace transforms. *SIAM J. Sci. Stat. Comput.* 3, 3 (1982), 357–366. doi:10.1137/0903022

[7] H. Dubner and J. Abate. 1968. Numerical inversion of Laplace transforms by relating them to the finite Fourier cosine transform. *J. ACM* 15, 1 (1968), 115–123. doi:10.1145/321439.321446

[8] J. W. Goodman. 2005. *Introduction to Fourier Optics* (3rd ed.). Roberts & Company.

[9] M. Guizar-Sicairos and J. C. Gutiérrez-Vega. 2004. Computation of quasi-discrete Hankel transforms of integer order for propagating optical wave fields. *J. Opt. Soc. Am. A* 21, 1 (2004), 53–58. doi:10.1364/JOSAA.21.000053

[10] Fredrik Johansson and mpmath contributors. 2023. mpmath: a Python library for arbitrary-precision floating-point arithmetic, version 1.3.0. http://mpmath.org/

[11] Yumin Lin and Chuanju Xu. 2007. Finite difference/spectral approximations for the time-fractional diffusion equation. *J. Comput. Phys.* 225, 2 (2007), 1533–1552. doi:10.1016/j.jcp.2007.02.001

[12] A. F. Oskooi, D. Roundy, M. Ibanescu, P. Bermel, J. D. Joannopoulos, and S. G. Johnson. 2010. MEEP: A flexible free-software package for electromagnetic simulations by the FDTD method. *Comput. Phys. Commun.* 181, 3 (2010), 687–702. doi:10.1016/j.cpc.2009.11.008

[13] Gorgi Pavlov. 2026. Systematic parameter selection for FFT-based numerical inverse Laplace transform: CFL-informed tuning rules and quality diagnostics. *Chem. Eng. Sci.* 328 (2026), 123776. doi:10.1016/j.ces.2026.123776

[14] Zhi-Zhong Sun and Xiaonan Wu. 2006. A fully discrete difference scheme for a diffusion-wave system. *Appl. Numer. Math.* 56, 2 (2006), 193–209. doi:10.1016/j.apnum.2005.03.003

[15] A. Taflove and S. C. Hagness. 2005. *Computational Electrodynamics: The Finite-Difference Time-Domain Method* (3rd ed.). Artech House.

[16] A. Talbot. 1979. The accurate numerical inversion of Laplace transforms. *J. Inst. Math. Appl.* 23 (1979), 97–120. doi:10.1093/imamat/23.1.97

[17] B. E. Treeby and B. T. Cox. 2010. k-Wave: MATLAB toolbox for the simulation and reconstruction of photoacoustic wave fields. *J. Biomed. Opt.* 15, 2 (2010), 021314. doi:10.1117/1.3360308

[18] J. A. C. Weideman. 2006. Optimizing Talbot's contours for the inversion of the Laplace transform. *SIAM J. Numer. Anal.* 44, 6 (2006), 2342–2362. doi:10.1137/050625837

[19] K. S. Yee. 1966. Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media. *IEEE Trans. Antennas Propag.* 14, 3 (1966), 302–307. doi:10.1109/TAP.1966.1138693